

HW2_Report



Author: 111511198 鄭恆安

A. Concept and Code Detail of the MLP Model

1. MLPClassifier

Here is a summary of how the components fit together in the MLP model:

- **Initialization:** When an MLPClassifier object is created, the constructor initializes layers, activation functions, and optimization settings.
- **Forward Pass:** During training, the model computes predictions for the training data and potentially for validation data.
- **Backward Pass:** The gradients of the loss concerning weights and biases are computed, facilitating the adjustment of parameters.
- **Loss Calculation:** The binary cross-entropy loss is calculated to measure model performance.
- **Update Mechanism:** Using the chosen optimizer, weights and biases are updated based on the computed gradients.
- **Training Loop:** The model iterates over epochs, adjusting learning rates, logging loss information, and implementing early stopping as necessary.

Initialization

- **layers:** Defines the structure of hidden layers.
- **activate_function:** Specifies the activation function used in the neurons (e.g., sigmoid).
- **activate_derivative:** The derivative of the activation function, used during backpropagation.
- **optimizer:** Determines the optimization algorithm (e.g., Adam, SGD).
- **learning_rate:** Controls the step size during optimization.
- **n_epoch:** Number of training iterations.

```
def __init__(self, layers, activate_function, activate_derivative, optimizer, learning_rate=0.05, n_epoch=10000):  
    self.hidden_layers = layers  
    self.activate_function = activate_function  
    self.activate_derivative = activate_derivative  
    self.optimizer = optimizer
```

```
self.learning_rate = learning_rate
self.n_epoch = n_epoch
```

Forward Pass

The forward pass is responsible for computing the output of the network given an input. It involves:

- Calculating the linear combination of inputs and weights, adding biases, and applying the activation function for each layer.
- The method can operate in two modes: training (updating the activations) and validation (using a separate list to store activations).

```
def forwardPass(self, X, validate=False):
    """ Forward pass of MLP """
    if validate:
        a_list = [X] # Use a separate list for validation activations
    else:
        self.a = [X] # Keep the original activation list for training

    for i in range(len(self.W)):
        if validate:
            z = np.dot(a_list[-1], self.W[i]) + self.b[i]
        else:
            z = np.dot(self.a[-1], self.W[i]) + self.b[i]

        a = self.activate_function(z)

        if validate:
            a_list.append(a) # Append to the validation list
        else:
            self.a.append(a) # Append to the training list

    return a_list[-1] if validate else self.a[-1]
```

Backward Pass

The backward pass computes the gradients of the loss function with respect to the weights and biases, using the chain rule:

- It starts from the output layer, calculating the error.
- Gradients for each layer are calculated in reverse order, considering the activation function's derivative.

```
def backwardPass(self, y):
    """ Backward pass of MLP """
    # Compute the gradients of the loss with respect to the weights and b
    iases
    m = y.shape[0]
    dz = self.a[-1] - y.reshape(-1, 1)
    self.dW = []
```

```

self.db = []
for i in reversed(range(len(self.W))):
    dw = np.dot(self.a[i].T, dz) / m
    db = np.sum(dz, axis=0, keepdims=True) / m
    self.dW.insert(0, dw)
    self.db.insert(0, db)
    if i > 0:
        # compute the gradient of the loss with respect to the activa
tions
        dz = np.dot(dz, self.W[i].T) * self.activate_derivative(self.
a[i])

```

Parameter Update

The update method adjusts the weights and biases using the computed gradients and the specified optimizer. It includes implementations for various optimizers.

```

def update(self):
    """ The update method to update parameters """
    # Update the weights and biases
    if self.optimizer == optimizer.Adam:
        if not hasattr(self, 'm'):
            self.m = [np.zeros_like(w) for w in self.W]
            self.v = [np.zeros_like(w) for w in self.W]
            self.t = 0
        self.W, self.b, self.m, self.v, self.t = self.optimizer(self.W, s
elf.b, self.dW, self.db, self.learning_rate, self.m, self.v, self.t)
    elif self.optimizer == optimizer.SGD:
        self.W, self.b = self.optimizer(self.W, self.b, self.dW, self.db,
self.learning_rate)
    elif self.optimizer == optimizer.Momentum:
        if not hasattr(self, 'velocity'):
            self.velocity = [np.zeros_like(w) for w in self.W]
        self.W, self.b, self.velocity = self.optimizer(self.W, self.b, se
lf.dW, self.db, self.learning_rate, self.velocity)

```

Loss Function

The binary cross-entropy loss function is used to quantify the difference between the predicted output and the true labels. This function is suitable for binary classification problems.

```

def binaryCrossEntropy(self, y_hat, y):
    # add an epsilon to avoid taking the log of zero
    m = y.shape[0]
    epsilon = 1e-8
    loss = -1/m * np.sum(y * np.log(y_hat + epsilon)
        loss += (1 - y) * np.log(1 - y_hat + epsilon))

```

```
return loss
```

Training Process

The `fit` method orchestrates the training loop:

- It splits the dataset into training and validation sets.
- Initializes weights using **He or Xavier initialization** methods, which are effective for different activation functions and promote effective learning, enabling faster convergence and better generalization than **zero initialization**.
- Performs the forward pass, computes the loss, executes the backward pass, and updates parameters iteratively for a specified number of epochs.
- Implements early stopping based on validation loss to prevent overfitting.

```
def fit(self, X_train, y_train):
    """ Fit method for MLP, call it to train your MLP model """
    data_len = X_train.shape[0]
    # Split the data into training and validation sets
    split = int(data_len * 0.8)
    X_val, y_val = X_train[split:], y_train[split:]
    X_train, y_train = X_train[:split], y_train[:split]

    # Initialize weights and biases
    self.W = []
    self.b = []
    self.val_loss = []
    self.layers = [X_train.shape[1]] + self.hidden_layers + [1]
    y_train = np.array(y_train).flatten() # make sure y is one dimension

    # Weight initialization should be based on the layer size
    initialize_method = ['he', 'xavier']
    initialization = initialize_method[1]

    if initialization == 'he':
        # He initialization
        for i in range(len(self.layers) - 1):
            self.W.append(np.random.randn(self.layers[i], self.layers[i + 1]) * np.sqrt(2.0 / self.layers[i]))
            self.b.append(np.zeros((1, self.layers[i + 1])))
    else:
        # Xavier initialization
        for i in range(len(self.layers) - 1):
            self.W.append(np.random.randn(self.layers[i], self.layers[i + 1]) * np.sqrt(1.0 / self.layers[i]))
            self.b.append(np.zeros((1, self.layers[i + 1])))

    # Training loop
    for epoch in range(self.n_epoch):
```

```

        # Forward pass
        y_hat = self.forwardPass(X_train)
        # Compute loss (binary cross-entropy)
        loss = self.binaryCrossEntropy(y_hat, y_train)

        # Validation loss
        if(epoch%100==0):
            if X_val is not None and y_val is not None:
                y_val_hat = self.forwardPass(X_val, True)
                val_loss = self.binaryCrossEntropy(y_val_hat, y_val)
                self.val_loss.append(val_loss)

            if(epoch%(self.n_epoch/5)==0):
                self.learning_rate*=0.9
            if(epoch%(self.n_epoch/10)==0):
                print(f'Epoch {epoch + 1}/{self.n_epoch}, Loss: {loss}, Val Loss: {val_loss if X_val is not None else "N/A"}')

        # Backward pass
        self.backwardPass(y_train)

        # Update weights and biases
        self.update()

        # Early stopping to prevent overfitting
        if len(self.val_loss) > 5 and all(self.val_loss[-i] > self.val_loss[-i-1] for i in range(1, 6)):
            print("Early stopping due to increase in validation loss at epoch = ", epoch)
            break

```

2. Activation Functions

Activation functions introduce non-linearities into the model, enabling it to learn complex relationships. In class activation, I implement both functions and its derivative include:

- **ReLU (Rectified Linear Unit):** Sets any negative value to zero while keeping positive values unchanged. It is computationally efficient and reduces the likelihood of vanishing gradients.

```

@staticmethod
def relu(x):
    """ The relu function """
    return np.maximum(0, x)

@staticmethod
def relu_derivative(x):

```

```

""" Derivative of the relu function """
return np.where(x > 0, 1, 0)

```

- **Sigmoid:** Squashes output values between 0 and 1, which is useful for binary classification tasks but can suffer from vanishing gradients in deep networks.

```

@staticmethod
def sigmoid(x):
    """ The sigmoid function """
    return 1.0 / (1.0 + np.exp(-x))

@staticmethod
def sigmoid_derivative(x):
    """ Derivative of the sigmoid function """
    sig = activation.sigmoid(x)
    return sig * (1.0 - sig)

```

- **Tanh:** Similar to sigmoid but outputs values between -1 and 1, mitigating some vanishing gradient issues.

```

@staticmethod
def tanh(x):
    """ The tanh function """
    return np.tanh(x)

@staticmethod
def tanh_derivative(x):
    """ Derivative of the tanh function """
    return 1.0 - np.tanh(x) ** 2

```

3. Optimizers

Optimizers adjust model weights to minimize loss during training. This MLP model used:

- **SGD (Stochastic Gradient Descent):** Iteratively updates weights based on gradients of small, randomly selected data subsets. While efficient, it may converge slowly without momentum.

```

def SGD(W, b, dW, db, learning_rate):
    for i in range(len(W)):
        W[i] -= learning_rate * dW[i]
        b[i] -= learning_rate * db[i]
    return W, b

```

- **Momentum:** Adds a fraction of the previous update vector to the current update, helping avoid local minima and accelerate convergence.

```

def Momentum(W, b, dW, db, learning_rate, velocity, beta=0.9):
    for i in range(len(W)):
        velocity[i] = beta * velocity[i] + (1 - beta) * dW[i]

```

```

        W[i] -= learning_rate * velocity[i]
        b[i] -= learning_rate * db[i]
    return W, b, velocity

```

- **Adam:** Combines momentum and adaptive learning rates for faster convergence, making it suitable for most tasks.

```

def Adam(W, b, dW, db, learning_rate, m, v, t):
    t += 1
    beta1, beta2, epsilon = 0.9, 0.999, 1e-8

    for i in range(len(W)):
        m[i] = beta1 * m[i] + (1 - beta1) * dW[i]
        v[i] = beta2 * v[i] + (1 - beta2) * (dW[i] ** 2)

        m_hat = m[i] / (1 - beta1 ** t)
        v_hat = v[i] / (1 - beta2 ** t)

        W[i] -= learning_rate * m_hat / (np.sqrt(v_hat) + epsilon)
        b[i] -= learning_rate * db[i]
    return W, b, m, v, t

```

Main.py

```

train_X, train_y, test_X, test_y = dataPreprocessing(0.1) # train, test dat
a should not contain index

model = MLPClassifier(layers= [20,10,5] , activate_function=activation.sigm
oid, activate_derivative=activation.sigmoid_derivative, optimizer=optimize
r.Adam, learning_rate=0.05, n_epoch = 100000)

model.fit(train_X, train_y)
pred = model.predict(test_X)

```

B. Questions

1. What is gradient explosion? How can it be solved?

Gradient explosion occurs when gradients accumulate excessively during backpropagation, leading to large updates that destabilize the network. This issue often arises in deep networks with long sequences. To address gradient explosion:

- **Gradient clipping** limits gradients within a specific range, so that the gradient will not become very large.
- **Weight regularization** controls the magnitude of weights, preventing large values.
- **Alternative activation functions** like ReLU can mitigate gradient explosion by reducing the chances of extreme gradients.

2. What is overfitting? How can it be solved?

Overfitting is when a model learns the training data too well, capturing noise and reducing generalizability to new data. Solutions for overfitting include:

- **Regularization** (e.g., L1, L2): Adds a penalty for large weights, encouraging the model to learn simpler patterns.
- **Dropout**: Randomly "drops" neurons during training to prevent reliance on specific patterns.
- **Cross-validation**: Validates the model on multiple subsets, improving generalization by avoiding reliance on any single data partition.

In this homework, I use **Early Stopping Method**. By continuously monitoring validation loss and terminating training when performance deteriorates, it helps maintain a balance between model complexity and generalization, ultimately leading to better performance on unseen data.

C. Discussion

1. Activation Functions

Comparison without activation functions: Without activation functions, the MLP model becomes a linear model, losing the ability to capture complex patterns. Activation functions introduce non-linearities, making them essential for effective neural networks.

2. Comparison of Activation Functions

Only activation function will change, other parameters will remain the same.

And I find that using **sigmoid** as activation will get the best score.

```
layers= [20,10],
optimizer=optimizer.Adam,
learning_rate=0.05,
n_epoch = 100000
```

- **Sigmoid**

```
activate_function=activation.sigmoid,
activate_derivative=activation.sigmoi
d_derivative
```

```
Acc: 0.80233
F1 score: 0.70175
MCC: 0.55626
Scoring: 0.68100
```

- **ReLU**


```
activate_function=activation.relu,  
activate_derivative=activation.relu_derivative
```

```
Acc: 0.70930  
F1 score: 0.46809  
MCC: 0.31059  
Scoring: 0.48533
```

- **Tanh**

```
activate_function=activation.relu,  
activate_derivative=activation.relu_derivative
```

```
Acc: 0.65116  
F1 score: 0.51613  
MCC: 0.24416  
Scoring: 0.46145
```

3. Comparison of Optimizers

Only optimizer will change, other parameters will remain the same.

And I find that using **Adam** as optimizer will get the best score.

```
layers= [20,10],  
activate_function=activation.sigmoid,  
activate_derivative=activation.sigmoid_derivative  
learning_rate=0.05,  
n_epoch = 100000
```

- **SGD**

```
optimizer=optimizer.SGD
```

```
Acc: 0.76744  
F1 score: 0.67742  
MCC: 0.49653  
Scoring: 0.64111
```

- **Adam**

```
optimizer=optimizer.Adam,
```

```
Acc: 0.80233  
F1 score: 0.70175  
MCC: 0.55626  
Scoring: 0.68100
```

- Momentum

```
optimizer=optimizer.Momentum,
```

```
Acc: 0.76744  
F1 score: 0.67742  
MCC: 0.49653  
Scoring: 0.64111
```

D. Best Performance

```
layers= [20,10],  
activate_function=activation.sigmoid,  
activate_derivative=activation.sigmoid_derivative,  
optimizer=optimizer.Adam,  
learning_rate=0.05,  
n_epoch = 100000
```

```
Acc: 0.80233  
F1 score: 0.70175  
MCC: 0.55626  
Scoring: 0.68100
```

E. Bonus(different layers)

Only depth of layers will change, other parameters will remain the same.

```
activate_function=activation.sigmoid,  
activate_derivative=activation.sigmoid_derivative,  
optimizer=optimizer.Adam,
```

```
learning_rate=0.05,  
n_epoch = 100000
```

- depth = 2

```
layers = [20,10]
```

```
Epoch 1/100000, Loss: 347.35917655199484, Val Loss: 86.02985341984873  
Epoch 10001/100000, Loss: 501.22999540537546, Val Loss: 127.42905100197476  
Early stopping due to increase in validation loss at epoch = 13900  
[1 0 1 0 0 0 0 0 1 0 0 1 0 0 1 0 0 1 0 0 0 0 1 0 0 0 1 1 1 0 0 1 1 0 1 0 1  
 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 0 1 0 0 1 1 0 0 1 0 1 0 0 1 1 1 0 0 0 0 0  
 0 0 0 0 1 0 0 0 0 0 1 0]  
Acc: 0.80233  
F1 score: 0.70175  
MCC: 0.55626  
Scoring: 0.68100
```

- depth = 3

```
layers = [20,10,5]
```

```
Epoch 1/100000, Loss: 343.80723471401035, Val Loss: 85.94628524356735  
Early stopping due to increase in validation loss at epoch = 7600  
[1 0 0 0 0 0 0 0 1 0 0 1 0 0 1 0 0 1 0 0 0 0 1 0 0 0 1 0 0 0 0 1 1 0 1 0 1  
 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 1 0 0 1 1 0 0 1 0 1 0 0 1 1 1 0 1 0 0 0  
 0 0 0 0 1 0 0 0 0 0 1 0]  
Acc: 0.81395  
F1 score: 0.70370  
MCC: 0.57808  
Scoring: 0.69281
```

- depth = 5

```
layers = [30,25,20,10,5]
```

```
Epoch 1/100000, Loss: 359.3428902690815, Val Loss: 91.81799205426603  
Early stopping due to increase in validation loss at epoch = 9600  
[1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
 1 1 1 1 1 1 1 1 1 1 1 1]  
Acc: 0.37209  
F1 score: 0.52632  
MCC: 0.11294  
Scoring: 0.33537
```

◦

- depth = 10

```
layers = [30,25,20,20,15,15,10,5,3,1]
```

```
Epoch 1/100000, Loss: 360.55934140981987, Val Loss: 88.26971483030276  
Epoch 10001/100000, Loss: 343.76471035002805, Val Loss: 86.03190781970851  
Epoch 20001/100000, Loss: 343.7647103500281, Val Loss: 86.03190781970847  
Epoch 30001/100000, Loss: 343.76471035002817, Val Loss: 86.0319078197084  
Epoch 40001/100000, Loss: 343.7647103500281, Val Loss: 86.03190781970856  
Early stopping due to increase in validation loss at epoch = 45100  
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  
 0 0 0 0 0 0 0 0 0 0 0 0 0]  
Acc: 0.65116  
F1 score: 0.00000  
MCC: 0.00000  
Scoring: 0.19535
```

Findings

According to the experiment result above, we can find that when data is limited, simpler networks tend to perform better.

With increasing depth:

- **Learning becomes harder** due to the difficulty in adjusting a higher number of parameters effectively.
- **Overfitting increases** as the model has more capacity to memorize rather than generalize. **Increased capacity** may lead to overfitting, as the model learns noise or irrelevant details rather than general patterns.
- **Optimization becomes harder** with more layers, often causing gradient-related issues (like vanishing or exploding gradients) that impact learning during back propagation.

Thus, with limited data, **shallower networks (depth = 2 or 3) are preferable**, as they balance model capacity and learning stability.